8 Queens Backtracking Approach

Play the game: https://www.brainbashers.com/queens.asp

📌 Goal:

Place 8 queens on an 8×8 chessboard such that **no two queens attack each other**:

- Not in the same row
- Not in the same column
- Not in the same diagonal

☑ Backtracking Algorithm (Conceptual Steps)

1. Start from the **first row**.
2. Try placing a queen in **each column** of the current row.
3. For each placement, check if it's **safe** (no queen in same column or diagonal).
4. If safe:
    - Place the queen.
    - Recurse to the **next row**.
5. If all 8 queens are placed, you've found a **solution**.
6. If no valid column is found in a row, **backtrack** to the previous row and try the next column.

📦 Time Complexity

- Worst-case time complexity: **O(N!)**
- But backtracking **prunes** a large portion of the search space.

📝 Notes

- You can modify `solve_n_queens()` to **store all solutions** instead of stopping after the first one.
- This can be generalized to **N-Queens** by changing `N`.
- Total possible solutions: 92

Python Code for 8 Queens Using Backtracking

```
N = 8  # Size of the board

def print_board(board):
    for row in board:
        print(" ".join("Q" if col else "." for col in row))
    print()

def is_safe(board, row, col):
    # Check column
    for i in range(row):
        if board[i][col]:
            return False

    # Check upper-left diagonal
    for i, j in zip(range(row - 1, -1, -1), range(col - 1, -1, -1)):
        if board[i][j]:
            return False

    # Check upper-right diagonal
    for i, j in zip(range(row - 1, -1, -1), range(col + 1, N)):
        if board[i][j]:
            return False

    return True

def solve_n_queens(board, row):
    if row == N:
        print_board(board)
        return True  # If you want all solutions, remove this return and continue

    for col in range(N):
        if is_safe(board, row, col):
            board[row][col] = 1  # Place queen
            if solve_n_queens(board, row + 1):
                return True
            board[row][col] = 0  # Backtrack

    return False

# Initialize board
board = [[0 for _ in range(N)] for _ in range(N)]

# Solve the problem
if not solve_n_queens(board, 0):
    print("No solution found.")
```